Theoretical Computer Science Exam, January 24, 2020

The exam consists of **4 exercises**. Available time: 1 hr 40 min.; you may write your answers in Italian, if you wish.

All exercises must be addressed for the written exam to be considered sufficient.

F.NAME …….. L.NAME …….. MATRICOLA ……

I have passed the midterm exam and I intend to skip the first part (Ex.1-2) (select one): **YES NO**

Exercise 1 (8 pts)

Consider a given alphabet Σ. Given any two strings $x, y \in \Sigma^*$, let us say that y is a substring of x iff y can be obtained from x by removing any of its characters (also all or none of them). So, for instance, the substrings of $x = abaa$ are exactly the following: ε, *a*, *b*, *ab*, *ba*, *aa*, *aba*, *aaa*, *baa*, *abaa*.

Given any language L on alphabet Σ, that is, $L \subseteq \Sigma^*$, let us define ***SUBSEQ(L)*** as the language that includes exactly the substrings of all the strings of L :

$$SUBSEQ(L) = \{ y \in \Sigma^* \mid y \text{ is a substring of } x, \text{ for some } x \in L \}$$

Now let us consider a unary alphabet Σ, that is, $|\Sigma| = 1$. Prove, or disprove, the following statement:

For ***every*** language L on a unary alphabet, $SUBSEQ(L)$ is a regular language.

Please be at the same time clear and concise in your answer. (for additional space, use the last page)

Exercise 2 (8 pts)

Write a Monadic First Order (or Monadic Second Order only if necessary) logic formula characterizing the following languages

1) $L_1 = \{ x \in \{a, b, c\}^* \mid x$ contains (at least) one pair of consecutive *a*'s $\}$ For instance, $cbcaab \in L_1$, $aba \notin L_1$, $cbaabcaab \in L_1$, $cbaaaab \in L_1$.
2) $L_2 = \{ x \in \{a, b, c\}^* \mid x$ contains no more than one pair of consecutive *a*'s $\}$ For instance, $cbcaab \in L_2$, $aba \in L_2$, $cbaabcaab \notin L_2$, $cbaaaab \notin L_2$.
3) $L_3 = \{ x \in \{a, b, c\}^* \mid x$ contains an odd number of pairs of consecutive *a*'s $\}$ For instance, $aabca \in L_3$, $aba \notin L_3$, $baaabc \notin L_3$, $cbaaaab \in L_3$. For this language, in addition, show the value of the predicates used in the formula at each position of the following words: *aabca*, *baaabc*, and *cbaaaab.*

Exercise 3 (8 pts)

Prove that the family of semidecidable sets is closed under intersection, i.e., for every set S_1 and S_2, both semidecidable, their intersection $S_{12} = S_1 \cap S_2$ is also semidecidable. To this end, answer the following questions.

1. Outline a procedure $semDec_{S12}$ that "semidecides" S_{12}, i.e., such that, for $x \in S_{12}$ $semDec_{S12}(x)$ terminates and returns true, while for $x \notin S_{12}$ $semDec_{S12}(x)$ does not terminate; you can assume the existence of similar procedures $semDec_{S1}$ and $semDec_{S2}$ that semidecide S_1 and S_2.
2. Outline a procedure that enumerates S_{12}.
3. Under the assumption that $S_{12} \neq \emptyset$, write a definition of the generating function g_{S12} of S_{12} (hint: exploit the fact that the generating functions g_{S1} of S_1 and g_{S2} of S_2 exist; make sure that g_{S12} is total computable and its image is S_{12}).

Exercise 4 (8 pts)

1. Describe a 1-tape Turing machine that takes as input a string of type $a^n b^m$, with $n = 3^k$, $k \geq m$, $m > 0$, and outputs a^h, with $h = \frac{3^k}{3^m} = 3^{k-m}$. For instance, when the input is *aaab* the output is *a*; when the input is $a^{27}b^2$ the outpt is a^3. For the sake of simplicity, ignore the case in which the input is not a string of the specified type. Analyze the time and space complexity of this Turing machine, still ignoring the case in which the input is not a string of the specified type.
2. Describe a single tape Turing machine that accepts the language $\{ a^n \mid n = 3^k, k \geq 0\}$. Analyze its time and space complexity.

Solutions

Exercise 1

The statement is true. Consider in fact any language L on a unary alphabet (e.g.: $\Sigma = \{a\}$). Two cases are given: either L is finite or it is infinite.

If L is *finite*, then the maximal length of a string in L is defined, i.e., $\max_{x \in L}|x| = n$ for some $n \in N$. Since the alphabet is unary, $a^n \in L$. Clearly the substrings of a^n are the set $S = \{ a^k \mid 0 \le k \le n \}$, and no other string are included in $SUBSEQ(L)$, that is, $S = SUBSEQ(L)$. Hence $SUBSEQ(L)$, being a finite set, is necessarily regular.

On the other hand, if L is *infinite*, it includes arbitrarily large strings, that is, for every $n \in N$, there exists a number $n' > n$ such that $a^{n'} \in L$. Hence, for every $n \in N$ there exists a number $n' > n$ such that the set $\{ a^k \mid 0 \le k \le n' \}$ is a subset of $SUBSEQ(L)$. Therefore $SUBSEQ(L) = \{ a^k \mid k \in N \}$, and as a consequence $SUBSEQ(L)$ is the universal language Σ^* over alphabet Σ, and it is regular.

Truly, the same property ($SUBSEQ(L)$ is regular) holds for any language L over any finite alphabet Σ (of any cardinality). This property, whose proof is not trivial, is known as Higman's Lemma; see A. G. Higman: *Ordering by divisibility in abstract algebra*. Proceedings of the London Mathematical Society, 3:326-336, 1952.

Exercise 2

1) $L_1 = \{ x \in \{a, b, c\}^* \mid x$ contains (at least) one pair of consecutive a's $\}$ For instance, $cbcaab \in L_1$, $aba \notin L_1$, $cbaabcaab \in L_1$, $cbaaaab \in L_1$.

   $$\exists x \, (\neg last(x) \wedge a(x) \wedge a(x+1) \quad)$$

2) $L_2 = \{ x \in \{a, b, c\}^* \mid x$ contains no more than one pair of consecutive a's $\}$ For instance, $cbcaab \in L_2$, $aba \in L_2$, $cbaabcaab \notin L_2$, $cbaaaab \notin L_2$.

   $$\neg \, \exists x \, \exists y \, (\, y \neq x \; \wedge \; (\neg last(x) \wedge a(x) \wedge a(x+1)) \; \wedge \; (\neg last(y) \; \wedge \; a(y) \wedge a(y+1)) \quad)$$

3) $L_3 = \{ x \in \{a, b, c\}^* \mid x$ contains an odd number of pairs of consecutive a's $\}$ For instance, $aabca \in L_3$, $aba \notin L_3$, $baaabc \notin L_3$, $cbaaaab \in L_3$. For this language, in addition, show the value of the predicates used in the formula at each position of the following words: *aabca*, *baaabc*, and *cbaaaab.*

   Introducing:

   predicate $P(x) \leftrightarrow \neg last(x) \wedge a(x) \wedge a(x+1)$ meaning "at position x there are two consecutive occurrences of a's"

   predicate $X(x)$ meaning "up to position x included there is an odd number of consecutive occurrences of two a's"

   $$(X(0) \leftrightarrow P(0) \;) \; \wedge \; \forall z \, (\neg first(z) \rightarrow (X(z) \leftrightarrow (\; (X(z\text{-}1) \wedge \neg P(z)) \; \vee \; (\neg X(z\text{-}1) \wedge P(z) \;) \;) \;) \;)$$

   The MSO logic formula defining L_3 is

   $$\exists P \, \exists X \; (\, \forall x (P(x) \leftrightarrow \ldots) \quad \wedge \quad (X(0) \leftrightarrow P(0)) \wedge \forall z (\neg first(z) \rightarrow \ldots) \quad \wedge \quad \exists y \, (\, last(y) \wedge X(y) \,) \quad)$$

   Values of predicates P and X for strings *aabca*, *baaabc*, and *cbaaaab*

	aabca	baaabc	cbaaaab
P	TFFFF	FTTFFF	FFTTTFF
X	TTTTT	FTFFFF	FFTFTTT

Exercise 3

1. $semDec_{S12}$ for any input x alternately executes $semDec_{S1}$ and $semDec_{S2}$ for an increasing number of times; it stops and returns true iff both $semDec_{S1}$ and $semDec_{S2}$ stop and return true.
2. Just perform a dovetailing simulation of $semDec_{S12}$ using increasing values of its argument and the number of steps to be executed for each argument.
3. Consider (remember, in fact) the function $d(x, y)=(x+y)(x+y+1)/2+1$ that bijectively maps pairs of integers into an integer.
   Let $(x, y) = d^{-1}(n)$ and K = any element of S_{12} (which is not empty by hypothesis),
   then $g_{S12}(n)$ = if $g_{S1}(x)=g_{S2}(y)$ then $g_{S1}(x)$ (or equivalently $g_{S2}(y)$) else K

Exercise 4 (sketchy)

1. First, copy the initial part of the input (the sequence of *a*'s) on the memory tape. Then repeatedly, one time for each *b* in the input, cancel (e.g., by marking with a '*'), 2 *a*'s every 3 from the memory tape. Finally, copy on the output all the *a*'s remaining in the memory tape. Space complexity $S(n) = \Theta(n)$; time complexity is $T(n) = \Theta(n \cdot m) = \Theta(n \log n)$ in the worst case.
2. Similar to the previous point. Execute k scans of the input: (1) for every *a* in the input cancel (e.g., by marking with a '*'), the two following ones (skip any '*'); if these are not found, reject; (2) if only one *a* is left, accept, otherwise go back to step (1). Complexity: Let $n = |x| = 3^k$; then $S(n) = \Theta(n)$, and $T(n) = \Theta(n \log n)$.